

Projekt SWTM

Dokumentation

Aktivitätstracker

Android Application

Auftraggeber: pep.digital GmbH, Esslingen

Modul: Projektarbeit SWTM

Hochschule: Hochschule Esslingen

Semester: Sommersemester 2026

Teammitglieder

Name	Matrikelnummer
Daniel Trbara	779034
Christian Sakru	779980
Lionel Toussoumsnaba	781030
Mirac Sancak	781232
Abdullah Bilen	779202

Inhaltsverzeichnis

1	Meilenstein	1
1.1	Problemstellung und Zielgruppe	1
1.1.1	Problemstellung	1
1.1.2	Zielgruppe	1
1.2	Geforderter Funktionsumfang	2
1.2.1	Minimum Viable Product (MVP)	2
1.2.2	Erweiterte Ziele (Extended Goals)	2
1.2.3	Zielsetzung	3
1.3	UI-Entwürfe	4
1.4	Architektur (konzeptionell + technologisch)	9
1.4.1	Logische Sicht	9
1.4.2	Technischer Stack	10
1.4.3	Begründung der Technologiewahl	10
1.5	Projektmanagement	11
1.5.1	Ziel des Projektmanagements	11
1.5.2	Arbeitsweise	11
1.5.3	Kommunikations- und Organisationstools	12
1.6	Aufwandsschätzung	12
2	Meilenstein	13
2.1	User Stories	13
2.1.1	MVP	13
2.1.2	Extended	15
2.2	Die Softwarearchitektur und ihre logische Sichten	16
2.2.1	Die Struktursicht	16
2.2.2	Verhaltenssicht	17
2.2.3	Verteilungssicht	19
2.3	Verwendete Technologien und Frameworks	20
2.4	Verwendete Schnittstellentechnologie	22
2.5	Datenbank	23
2.5.1	Logisches Datenmodell	23
2.5.2	Physisches Datenmodell	24
	Anhang	25
	Quellen	25

1 Meilenstein

1.1 Problemstellung und Zielgruppe

1.1.1 Problemstellung

Viele Menschen verbringen einen Großteil ihres Alltags sitzend, sei es im Büro oder im Home-Office vor dem PC. Bewegung kommt dabei häufig zu kurz. Vor allem in den Wintermonaten verstärkt sich dieses Problem durch den Mangel an Sonnenlicht. Langes Sitzen ohne ausreichende Bewegung führt zu einer schlechten Körperhaltung, die langfristig die Muskulatur schwächt.

Kernproblem: Büroangestellte und Home-Office-Arbeitende bewegen sich im Alltag zu wenig, was zu gesundheitlichen Langzeitfolgen führt. Es fehlt ein niedrigschwelliges, motivierendes Werkzeug zur Integration von Alltagsbewegung.

Hier setzt der **Aktivitätstracker** an: Er soll Nutzerinnen und Nutzer motivieren, regelmäßig Outdoor-Aktivitäten durchzuführen. Dies wird durch ein Streak-System und tagesbasierte Erinnerungen unterstützt.

1.1.2 Zielgruppe

Die Anwendung richtet sich primär an folgende Nutzergruppe:

- Personen, die überwiegend sitzend im Büro oder Home-Office arbeiten
- Nutzer, die fit und gesund bleiben möchten, mit leichter regelmäßiger Outdoor-Aktivität (z. B. Spazierengehen, Radfahren)
- Die Zielgruppe ist **nicht** leistungsorientiert

1.2 Geforderter Funktionsumfang

1.2.1 Minimum Viable Product (MVP)

Das MVP beschreibt die grundlegenden Funktionen, die notwendig sind, um einen ersten nutzbaren Prototyp bereitzustellen.

Funktion	Beschreibung
Aktivitäts-Tracking	Nutzer können tägliche Aktivitäten (z. B. Gehen, Radfahren) erfassen.
Streak-System	Das System zählt aufeinanderfolgende aktive Tage. Bei fehlender Aktivität wird die Streak unterbrochen. Dient als Motivationsmechanismus.
Widgets	Home-Screen-Widgets ermöglichen schnelle Eingabe und direkte Streak-Anzeige ohne App-Öffnung.
Wetter/tagesbasierte Erinnerungen	Push-Benachrichtigungen basierend auf Uhrzeit oder Wetterbedingungen motivieren zur Aktivität bei günstigen äußeren Bedingungen.

1.2.2 Erweiterte Ziele (Extended Goals)

Die folgenden Funktionen erweitern den MVP und verbessern die Benutzererfahrung. Sie sind für die erste Version nicht zwingend erforderlich.

- **Statistiken:** Längste Streak, Wochen-/Monatsübersichten, Aktivitätsanalysen
- **Dark-/Light-Theme:** Umschaltbares Design nach Nutzerpräferenz
- **Mehrsprachigkeit:** Deutsch und Englisch als primäre Sprachen
- **Streaks mit Freunden teilen:** Soziale Komponente zur gegenseitigen Motivation
- **Öffentliche Challenges:** Gruppenherausforderungen zur Steigerung der Motivation durch Wettbewerb

1.2.3 Zielsetzung

Der Aktivitätstracker verfolgt konkrete, messbare Ziele, die über die allgemeine Problemlösung hinausgehen und als Erfolgskriterien für das Projekt dienen:

- Nutzer sollen jeden Tag eine Outdoor-Aktivität erfassen können
- Eine Aktivität soll innerhalb von wenigen Sekunden erfassbar sein
- Wetter- und tagesbasierte Benachrichtigungen sollen automatisch bei geeigneten Bedingungen ausgelöst werden
- Das Streak-System soll die tägliche Nutzungsfrequenz messbar erhöhen, ohne dabei leistungsorientierten Druck aufzubauen

Auf Basis dieser Ziele wurden die folgenden Architektur- und Technologieentscheidungen getroffen, die in Abschnitt [1.4](#) beschrieben werden.

1.3 UI-Entwürfe

Die folgenden Abbildungen zeigen die Entwürfe der Anwendung. Die Bildunterschriften sind jeweils mit dem Quellenverzeichnis verlinkt. Alle UI-Entwürfe wurden von Lionel Toussoumsnaba in Figma entwickelt (siehe Quelle [\[1\]](#)).

Der Hauptbildschirm (siehe Abbildung 1.1) zeigt die aktuelle Streak-Anzeige des Nutzers. Die Streak-Visualisierung ist von modernen Tracking-Apps wie YAZIO oder Duolingo inspiriert.

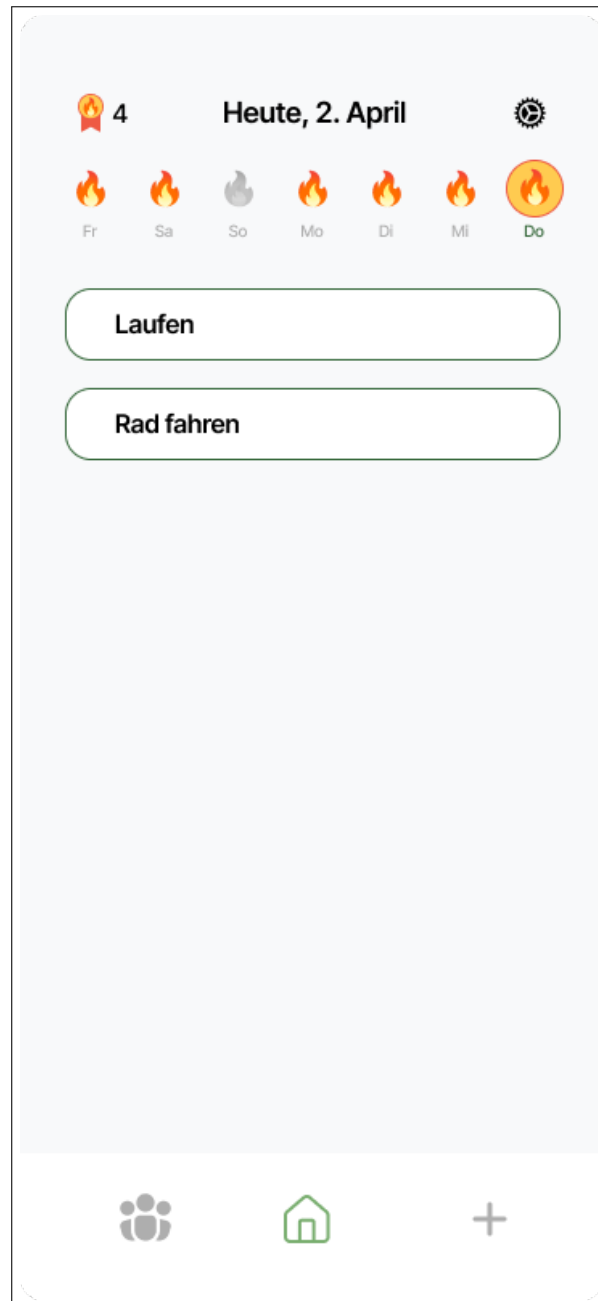


Abbildung 1.1: [Home Screen: Hauptmenü mit Streak-Anzeige](#)

Die Registrierung (siehe Abbildung 1.2) ermöglicht die erstmalige Anmeldung per E-Mail oder alternativ über ein Google-Konto. Der Anmeldebildschirm (siehe Abbildung 1.3) bietet dieselben Login-Optionen für bereits registrierte Nutzer.

The registration screen features the 'Steady' logo at the top. Below it, the heading 'Registrierung' is followed by the text 'Willkommen! Registriere dich, um fortzufahren.' The form includes three input fields: 'Username', 'E-Mail', and 'Passwort'. Below these fields is a button labeled 'Mit Google anmelden' with the Google logo. At the bottom is a large green button labeled 'Registrieren'.

Abbildung 1.2: Registrierung: Übersicht zur erstmaligen Anmeldung oder alternativen Google-Login

The login screen features the 'Steady' logo at the top. Below it, the heading 'Anmeldung' is followed by the text 'Melde dich an, um fortzufahren.' The form includes two input fields: 'E-Mail' and 'Passwort'. Below these fields is a button labeled 'Mit Google anmelden' with the Google logo. At the bottom is a large green button labeled 'Anmelden'.

Abbildung 1.3: Anmeldebildschirm: Login via E-Mail und Passwort oder alternatives Google-Login

Neue Aktivitäten können über ein Modal erfasst werden (siehe Abbildung 1.4). Bestehende Einträge lassen sich anschließend über einen separaten Bearbeitungsscreen anpassen (siehe Abbildung 1.5).

Das Modal 'Aktivität erstellen' ist ein vertikales Formular mit einem hellgrauen Hintergrund. Oben links befindet sich ein schwarzes 'X' in einem Quadrat, gefolgt vom Titel 'Aktivität erstellen' in einer fetten, schwarzen, sans-serif Schrift. Darunter befinden sich zwei Eingabefelder. Das obere Feld ist mit 'Name' beschriftet und enthält den Text 'Ins Gym gehen'. Das untere Feld ist mit 'Datum' beschriftet und enthält das Datum '08.06.2026'. Am unteren Rand des Modals befindet sich ein breiter, abgerundeter Button in einem dunkelgrünen Farbton mit der Aufschrift 'Speichern' in weißer, kleinerer, sans-serif Schrift.

Abbildung 1.4: Aktivität erstellen: Modal zur Eingabe und Speicherung einer neuen Aktivität

Das Modal 'Aktivität bearbeiten' ist ein vertikales Formular mit einem hellgrauen Hintergrund. Oben links befindet sich ein schwarzes 'X' in einem Quadrat, gefolgt vom Titel 'Aktivität bearbeiten' in einer fetten, schwarzen, sans-serif Schrift. Darunter befinden sich zwei Eingabefelder. Das obere Feld ist mit 'Name' beschriftet und enthält den Text 'Rad fahren'. Das untere Feld ist mit 'Datum' beschriftet und enthält das Datum '08.06.2026'. Am unteren Rand des Modals befindet sich ein breiter, abgerundeter Button in einem dunkelgrünen Farbton mit der Aufschrift 'Speichern' in weißer, kleinerer, sans-serif Schrift.

Abbildung 1.5: Aktivität bearbeiten: Modal zur Anpassung einer bereits bestehenden Aktivität

Im Einstellungsbereich (siehe Abbildung 1.6) können Nutzer Benachrichtigungszeiten und Kontodetails verwalten. Die Profileinstellungen (siehe Abbildung 1.7) ermöglichen zusätzlich die Änderung des Benutzernamens.

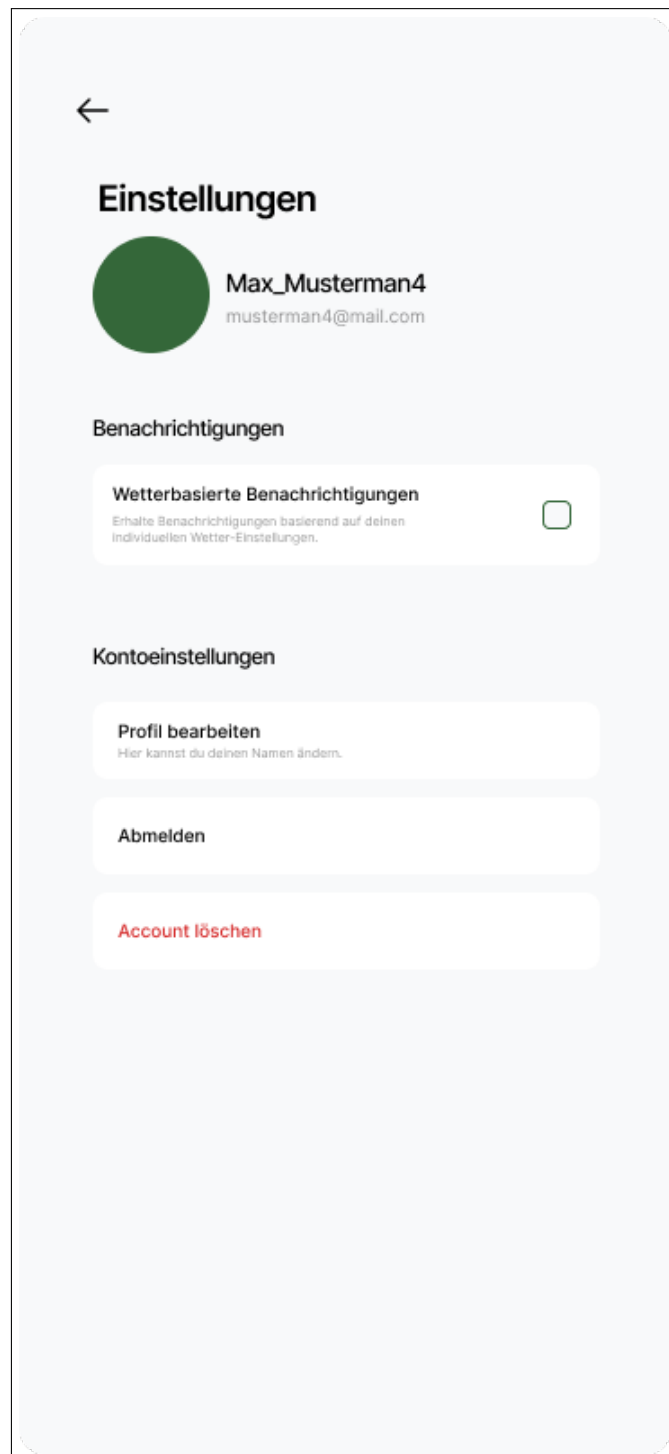


Abbildung 1.6: Einstellungen: Verwaltung von Benachrichtigungen und Kontodetails

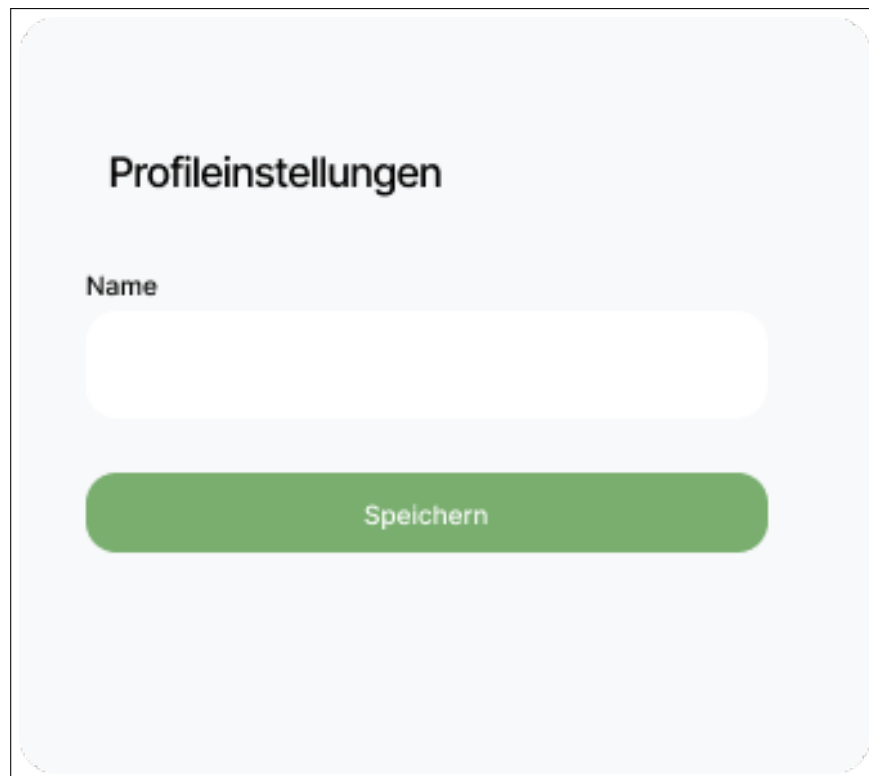


Abbildung 1.7: Profileinstellungen: Menü zur Änderung des Benutzernamens

Push-Benachrichtigungen (siehe Abbildung 1.8) informieren den Nutzer bei Inaktivität sowie bei erreichten Streak-Meilensteinen. Das Home-Screen-Widget (siehe Abbildung 1.9) ermöglicht einen schnellen Überblick über die wöchentliche Streak ohne die App öffnen zu müssen.

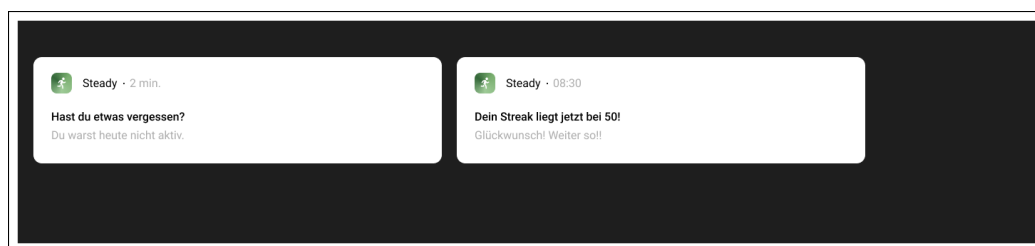


Abbildung 1.8: Benachrichtigungen: Darstellung von Inaktivitäts-Erinnerungen und Streak-Meilensteinen

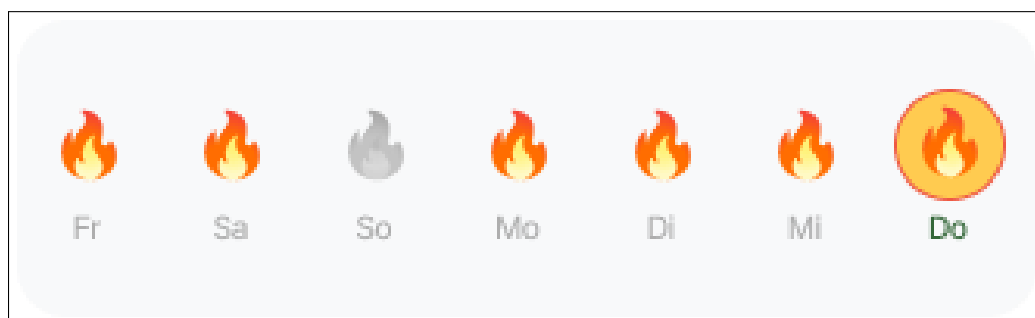


Abbildung 1.9: Streak-Widget: Isolierte Ansicht der wöchentlichen Streak-Visualisierung

1.4 Architektur (konzeptionell + technologisch)

1.4.1 Logische Sicht

Die Anwendung ist in drei Schichten aufgeteilt: UI-Schicht, Logik-Schicht und Datenzugriff. Wie in Abbildung 1.10 dargestellt, kommuniziert die UI-Schicht (Jetpack Compose) ausschließlich über die Logik-Schicht mit dem Datenzugriff. Der Datenzugriff kommuniziert sowohl mit der lokalen Room-Datenbank als auch mit dem Spring Boot Backend und der WeatherAPI über HTTP.

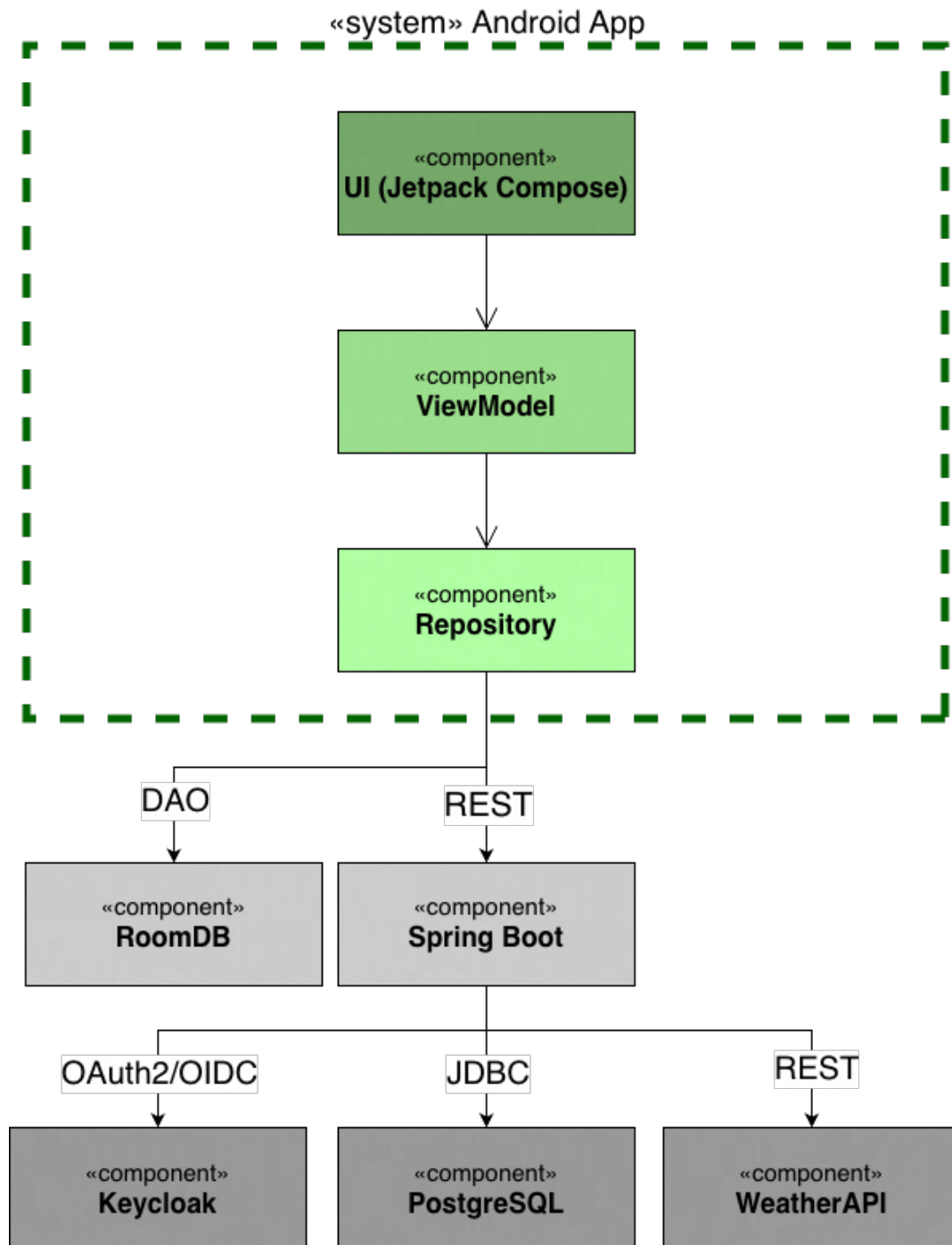


Abbildung 1.10: Komponentenmodell

1.4.2 Technischer Stack

Der technische Stack (siehe Tabelle 1.1) umfasst alle eingesetzten Technologien und Bibliotheken der Anwendung. Die Begründung der einzelnen Technologieentscheidungen ist in Abschnitt 1.4.3 beschrieben.

Schicht	Technologie / Bibliothek
Frontend	Android App + Jetpack Compose (Kotlin)
Architektur	Schichtenbasierter Ansatz (UI, Logik, Datenzugriff)
Lokale DB	Room (SQLite) – Offline-Verfügbarkeit
API-Client	Retrofit 2 – REST-Kommunikation
Backend	Spring Boot – REST API und Datenverwaltung
Authentifizierung	Keycloak
Datenbank	PostgreSQL
Wetter-API	WeatherAPI.com – wetterbasierte Erinnerungen
Widgets	Glance API (Jetpack)
Versionskontrolle	GitHub ¹
Build-System (Android)	Gradle (Kotlin DSL)
Build-System (Backend)	Maven

Tabelle 1.1: Technischer Stack des Projekts

1.4.3 Begründung der Technologiewahl

Die Auswahl der eingesetzten Technologien basiert sowohl auf Vorgaben des Auftraggebers als auch auf technischen und wirtschaftlichen Überlegungen des Teams.

Keycloak wurde als Authentifizierungslösung gewählt, da es vom Auftraggeber pep.digital und der Hochschule vorgegeben wurde und den OAuth2/OIDC-Standard vollständig implementiert.

PostgreSQL wird serverseitig als primäre Datenbank eingesetzt, da Nutzerdaten und Aktivitäten zentral gespeichert werden müssen, um geräteübergreifenden Zugriff zu ermöglichen. Room dient lediglich als lokaler Cache für Offline-Verfügbarkeit und als Ergänzung zu PostgreSQL.

Spring Boot wurde als Backend-Framework gewählt, da es vorgegeben wurde und für die Tracking-App mit den anderen eingesetzten Technologien kompatibel ist.

Jetpack Compose ist das moderne UI-Framework für Android und wird von Google als offizieller Nachfolger des klassischen XML-basierten View-Systems empfohlen. Es wurde vom Auftraggeber vorgegeben und ermöglicht eine deklarative UI-Entwicklung,

¹Projekt-Repository: <https://github.com/hse-classes/ss26-aktivitatstracker>

bei der die Oberfläche direkt den aktuellen Zustand der Anwendung widerspiegelt. Gegenüber dem klassischen View-System entfällt unnötiger Overhead im Code.

Glance API wurde für die Umsetzung der Widgets gewählt, da sie eine moderne und deklarative Entwicklung von Android-Widgets ermöglicht. Die API reduziert den Implementierungsaufwand und verbessert die Wartbarkeit des Codes. Zudem passt Glance gut zum Jetpack-/Compose-Ansatz und eignet sich besonders für einfache, schnell zugängliche Informationen wie Aktivitäten und Streaks.

Retrofit ist bekannt dafür, Netzwerkanfragen für Android- und Java-Entwicklungen in wenigen Zeilen Code abzubilden. Es wurde ausgewählt, um die Verwaltung von REST-Calls zu vereinfachen und den Code lesbarer zu gestalten.

WeatherAPI.com wurde ausgewählt, da der kostenlose Tarif mit 100.000 Anfragen pro Monat für den Projektumfang ausreichend ist. Die Integration erfolgt via Retrofit, wobei der API-Key sicher im Backend verwaltet wird. Ein Vorteil ist die Rückgabe der Daten als strukturiertes JSON, was eine direkte Verarbeitung im Projekt ermöglicht (siehe Quelle [2]).

Room wurde gewählt, da es als offizieller Google-Standard gut dokumentiert ist und Datenbankfehler bereits beim Kompilieren erkennt. Die direkte Umwandlung von Datenbankeinträgen in Kotlin-Objekte reduziert den Implementierungsaufwand und verhindert typische Laufzeitfehler.

1.5 Projektmanagement

1.5.1 Ziel des Projektmanagements

Ziel ist eine strukturierte und nachvollziehbare Projektdurchführung in agilen Ein-Wochen-Sprints. Durch regelmäßige Meetings und konsequentes Issue-Tracking soll sichergestellt werden, dass Termine eingehalten und Fortschritte transparent dokumentiert werden.

1.5.2 Arbeitsweise

Das Team arbeitet nach einem Issue-basierten Ansatz ohne fest zugewiesene Rollen. Aufgaben werden als Issues im GitHub-Repository angelegt und vom Team untereinander aufgeteilt. Dies fördert flexible Zusammenarbeit und gleichmäßige Beteiligung aller Mitglieder. Das Projekt-Repository ist unter folgendem Link erreichbar: <https://github.com/hse-classes/ss26-aktivitatstracker>

1.5.3 Kommunikations- und Organisationstools

Zur Koordination und Kommunikation im Team werden die in Tabelle 1.2 aufgeführten Tools eingesetzt.

Tool	Verwendungszweck
GitHub	Versionskontrolle, Issue-Tracking, Kanban-Board
Discord	Team-Kommunikation
Figma	UI-Design, Prototypen
Overleaf	Dokumentierung
Microsoft Teams	Meetings mit Betreuer und Auftraggeber

Tabelle 1.2: Kommunikations- und Organisationstools

1.6 Aufwandsschätzung

Die folgende Tabelle (siehe Tabelle 1.6) zeigt die geschätzten Aufwände. Das Projekt ist in **Ein-Wochen-Sprints** gegliedert. Die Schätzung basieren auf Refinements der MVP's. Die Schätzungen wurden gemeinsam im Team festgelegt und orientieren sich an den angelegten Issues im GitHub-Repository.

Phasen	Ref. US	Gesamtaufwand (h)
Projektmanagement & Vorbereitung		
Sprint-Meetings, Ticket-Refinement & Organisation	–	210 h
Anforderungsanalyse, Architektur & UI-Design	–	160 h
Implementierung		
Setup (Backend, Keycloak, API)	1.x, 2.x	100 h
Aktivitäten-Tracking	3.x	150 h
Streak-System	4.x	60 h
Benachrichtigungs-Service & Wetter-API	5.x, 6.x	50 h
Widget (Glance API)	7.x	40 h
Qualitätssicherung & Abschluss		
Code-Reviews, Bugfixing & Testing	–	220 h
Dokumentation (MS-Doku, Präsi & Handbuch)	–	260 h
Gesamtaufwand geschätzt (Team)		1.250 h

Tabelle 1.3: Aufwandsschätzung

2 Meilenstein

2.1 User Stories

2.1.1 MVP

Registrieren

1.1 Als Benutzer möchte ich mich einmalig Registrieren können, damit ich Geräte übergreifend über meinen Fortschritt zugreifen kann.

Anmelden

2.1 Als Benutzer möchte ich mich anmelden können, damit ich wieder Zugriff auf meinen Account habe.

2.2 Als Benutzer möchte ich meinen Account löschen, damit alle meine Daten gelöscht werden.

Aktivitäten Tracken

3.1 Als Benutzer möchte ich eine Aktivität eintragen, damit ich mein Verhalten tracken kann.

3.2 Als Benutzer möchte ich meine Aktivitäten bearbeiten können, damit ich Fehler korrigieren kann.

3.3 Als Benutzer möchte ich meine Aktivitäten löschen können, damit ich falsche Einträge entfernen kann.

3.4 Als Benutzer möchte ich eine Liste meiner Aktivitäten sehen, damit ich den Überblick behalte.

3.5 Als Benutzer möchte ich meine Aktivitäten auch ohne Internetverbindung aufzeichnen, damit ich auch in einem Funkloch meine Aktivitäten Tracken kann.

3.6 Als Benutzer möchte ich den Namen der Aktivität nicht immer wieder aufschreiben müssen, sondern meine vorherigen Aktivitäten benutzen.

Streak verwalten

4.1 Als Benutzer möchte ich meinen aktuellen Streak sehen, damit ich motiviert bleibe.

4.2 Als Benutzer möchte ich eine Streak aufbauen können, damit ich einen Ansporn habe.

4.3 Als Benutzer möchte ich meine Streak verlieren können, damit ich bestraft werde.

4.4 Als Benutzer möchte ich meinen Streak-Verlauf sehen, damit ich meinen Fortschritt nachvollziehen kann.

Erinnerungen erhalten

5.1 Als Benutzer möchte ich Erinnerungen bekommen, damit ich meine Aktivität nicht vergesse.

5.2 Als Benutzer möchte ich Erinnerungen konfigurieren, damit ich zur passenden Zeit die richtige Motivation bekomme.

5.3 Als Benutzer möchte ich meine Erinnerung ändern können, damit sie besser zu meinem Tagesablauf passt.

Wetter basierte Erinnerungen erhalten

6.1 Als Benutzer möchte ich eine Erinnerung bekommen, wenn es draußen gutes Wetter ist.

Widgets nutzen

7.1 Als Benutzer möchte ich Aktivitäten über ein Widget eintragen, damit es schneller geht.

7.2 Als Benutzer möchte ich Widgets hinzufügen, damit ich meine Streaks schneller sehen kann.

7.3 Als Benutzer möchte ich ein Widget entfernen können, damit ich meinen Homescreen anpassen kann.

2.1.2 Extended

Statistiken checken

8.1 Als Benutzer möchte ich Statistiken sehen, damit ich meinen Fortschritt analysieren kann.

8.2 Als Benutzer möchte ich meine Statistiken nach Woche oder Monat filtern, damit ich meinen Fortschritt besser analysieren kann.

8.3 Als Benutzer möchte ich meinen längsten Streak sehen, damit ich meine beste Leistung kenne.

Theme ändern

9.1 Als Benutzer möchte ich zwischen Dark und Light Mode wechseln, damit ich die App angenehmer nutzen kann.

Sprache ändern

10.1 Als Benutzer möchte ich die App in verschiedenen Sprachen nutzen, damit ich sie besser verstehe.

Freund Streak ansehen

11.1 Als Benutzer möchte ich in der App meine Freunde und ihre Streaks sehen.

11.2 Als Benutzer möchte ich Freunde hinzufügen können, damit ich mich mit ihnen verbinden kann.

11.3 Als Benutzer möchte ich Freunde entfernen können, damit ich meine Liste verwalten kann.

Streak mit Freund teilen

12.1 Als Benutzer möchte ich meinen Streak mit Freunden teilen, damit ich mich austauschen kann

2.2 Die Softwarearchitektur und ihre logische Sichten

2.2.1 Die Struktursicht

Android App

Die Strukturschichten beschreiben die internen Bausteine des Systems und deren Beziehungen zueinander. Entwickelt wird die App mit Android Studio's Jetpack Compose; Ein UI-Framework Für die Darstellung der Nutzeroberfläche und Nutzerinteraktion.

Das **ViewModel** beinhaltet die Geschäftslogik, beispielsweise die Aktivitätsverwaltung und Streakberechnung.

Die **Repository** Kapselt den Datenzugriff und fungiert als Schnittstelle zwischen der Logik-Schicht und den Datenbanken.

RoomDB Dient als lokale SQLite-Datenbank zur Offline-Verfügbarkeit, auf die über Data Access Objects (DAO) zugegriffen wird.

Backend-Systeme

Spring Boot stellt das Backend dar, welches per REST angesprochen wird und die zentrale Datenverwaltung übernimmt.

PostgreSQL dient als relationale Datenbank zur persistenten und serverseitigen Speicherung. Die Anbindung an das Spring Boot Backend erfolgt über JDBC.

Keycloak ist für die Authentifizierung mittels OAuth2/OIDC-Standards zuständig.

Die genaue Anordnung und Kommunikation dieser gesamten Bausteine ist in Abbildung [2.1](#) visualisiert.

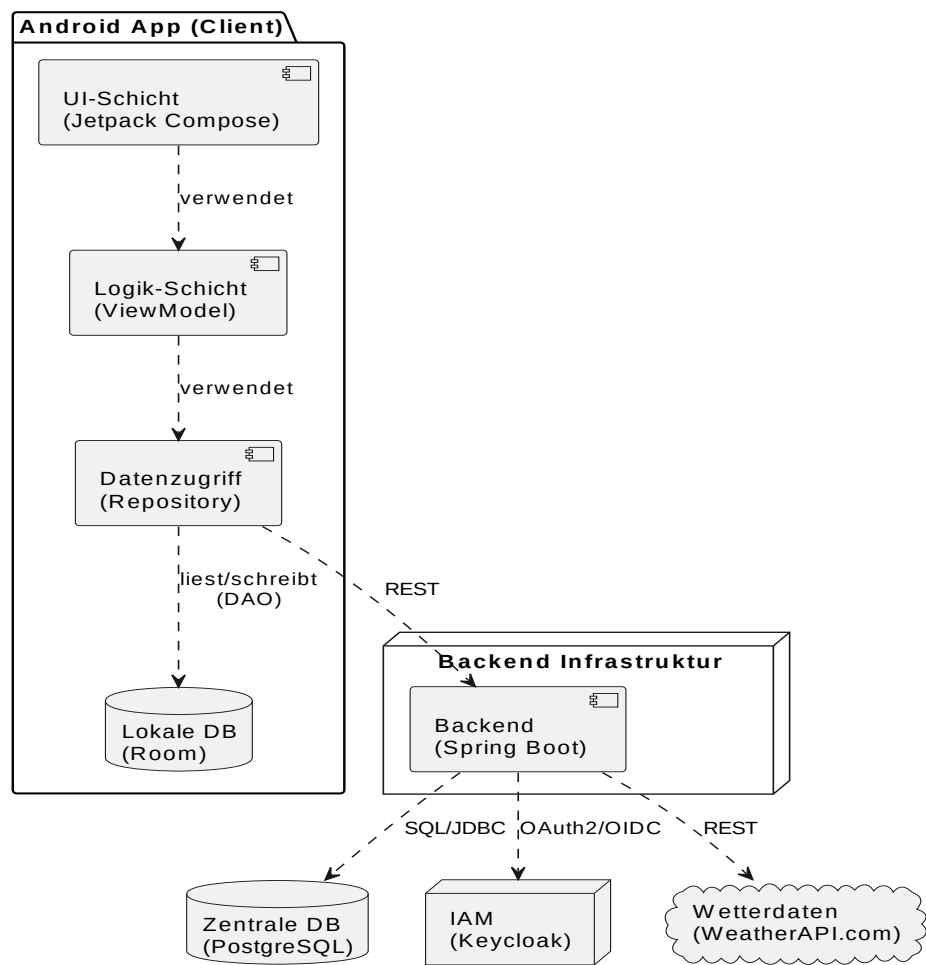


Abbildung 2.1: Komponentendiagramm der Strukturschicht

2.2.2 Verhaltenssicht

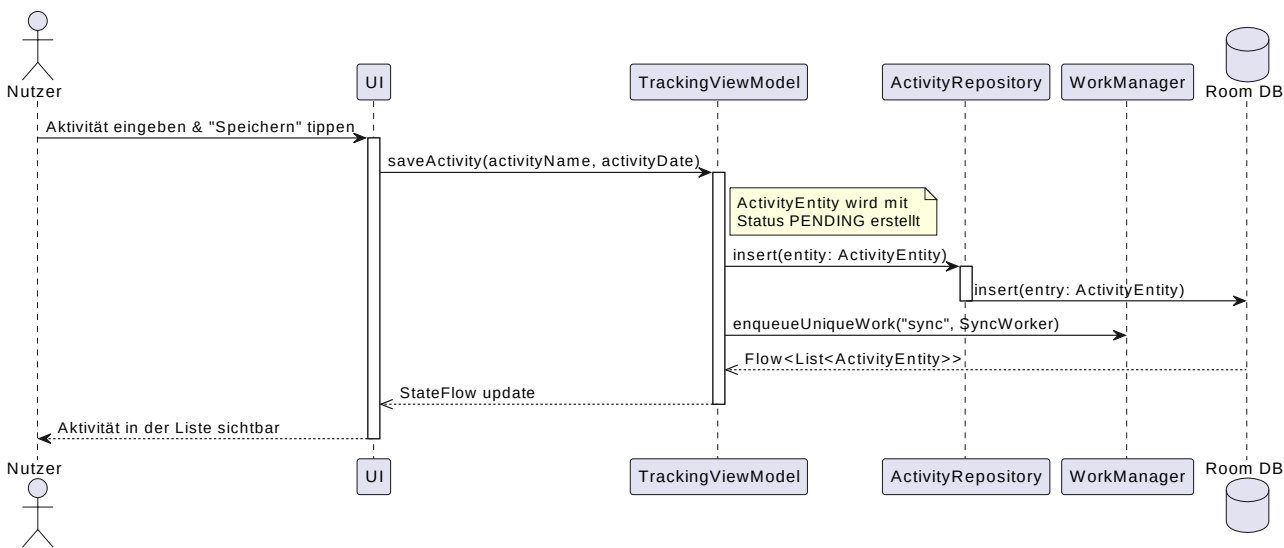


Abbildung 2.2: Prozess der Aktivitätserfassung

Die Grafik 2.2 veranschaulicht, wie die Softwarekomponenten zur Laufzeit miteinander interagieren, um die Kernanwendungsfälle abzuarbeiten.

Aktivität erfassen und speichern

Der Nutzer gibt eine neue Aktivität über ein Eingabefeld in der UI-Schicht ein. Die UI leitet die Eingabedaten an das TrackingViewModel weiter, welches eine ActivityEntity mit dem Status PENDING erstellt.

Das ViewModel ruft das Repository auf, welches die Entity über das DAO in der lokalen Room-Datenbank speichert. Dies gewährleistet eine sofortige Speicherung und Offline-Verfügbarkeit. Anschließend wird über den WorkManager ein Synchronisationsauftrag eingeplant.

Nach dem Speichern aktualisiert Room über einen reaktiven Datenstrom (State-Flow) automatisch die UI, sodass die neue Aktivität dem Nutzer unmittelbar angezeigt wird.

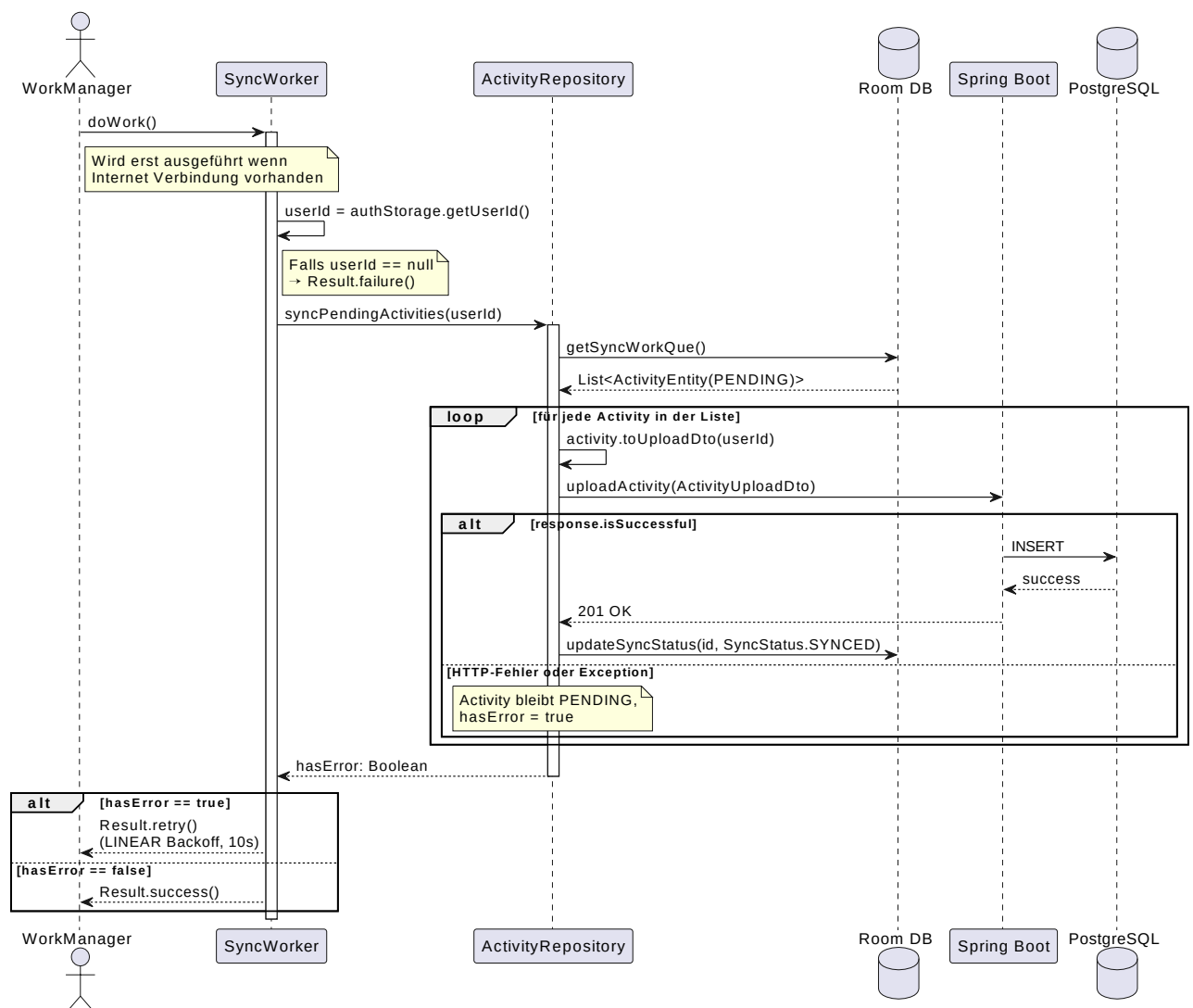


Abbildung 2.3: Prozess der Aktivitätssynchronisierung

Die Grafik 2.3 veranschaulicht, wie die Softwarekomponenten interagieren, um Aktivitätä-

ten synchronisieren zu können

Aktivitäten synchronisieren

Der WorkManager startet den SyncWorker, sobald eine Netzwerkverbindung verfügbar ist. Dieser ruft das Repository auf, welches alle Einträge mit dem Status PENDING aus der lokalen Room-Datenbank lädt.

Für jeden PENDING-Eintrag wird ein REST-Aufruf an das Spring Boot Backend gesendet. Das Backend speichert den Datensatz in der PostgreSQL Datenbank und bestätigt den Vorgang. Bei erfolgreicher Übertragung wird der lokale Status von PENDING auf SYNCED aktualisiert.

Falls ein Request fehlschlägt, bleibt der betroffene Eintrag auf PENDING und der SyncWorker plant automatisch einen erneuten Versuch ein.

Authentifizierung

Versucht der Nutzer sich über die App anzumelden (z. B. via E-Mail/Passwort oder Google-Login), kommuniziert das System mit Keycloak. Keycloak verifiziert die Identität per OAuth2/OIDC-Protokoll und autorisiert den sicheren Zugriff auf das Backend.

2.2.3 Verteilungssicht

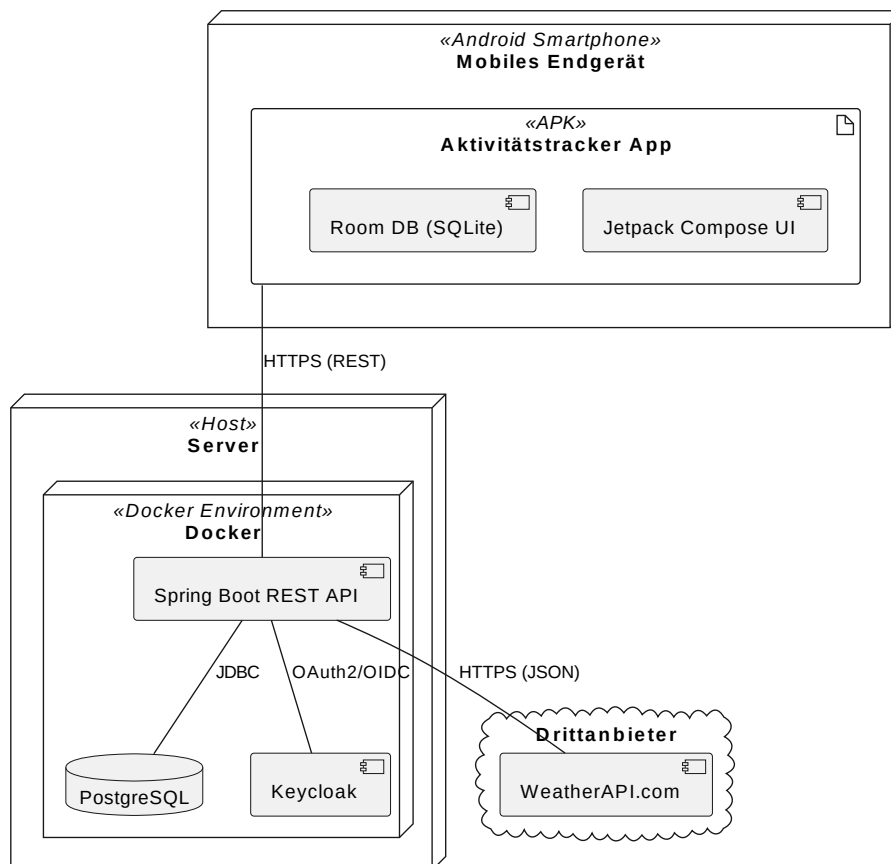


Abbildung 2.4: Deployment-Diagramm der Verteilungssicht

Die in Abbildung 2.4 abgebildete Verteilungssicht beschreibt die physische Verteilung der Softwarekomponenten auf die Hardware- und Netzwerk-Infrastruktur.

Client-Gerät (Android-Smartphone)

Auf dem Endgerät des Nutzers wird die kompilierte Android Application ausgeführt. Dieser Knoten beinhaltet das Frontend (Jetpack Compose) sowie die Room-Datenbank, welche als Single Source of Truth dient und Offline-Verfügbarkeit ermöglicht.

Server-Infrastruktur

Die serverseitigen Komponenten werden in einer gemeinsamen Docker-Umgebung auf einem einzelnen Server betrieben:

- **Spring Boot** stellt die zentrale REST-API für die App bereit.
- **PostgreSQL** speichert alle Nutzerdaten und Aktivitäten geräteübergreifend.
- **Keycloak** übernimmt die Authentifizierung mittels OAuth2/OIDC.

Zusätzlich werden über die externe Schnittstelle WeatherAPI.com Wetterdaten als strukturiertes JSON bezogen, um wetterbasierte Erinnerungen zu ermöglichen.

Die Kommunikation zwischen Android-App und Backend erfolgt momentan über HTTPS. In der aktuellen Entwicklungsumgebung werden selbstsignierte Zertifikate eingesetzt. Jedoch ist in Planung dies in HTTP umzuändern.

2.3 Verwendete Technologien und Frameworks

Im folgenden Abschnitt werden die verwendeten Technologien und Frameworks beschrieben, die im Projekt eingesetzt werden oder deren Einsatz geplant ist.

Frontend (Android)

Die Android-App wird vollständig in Kotlin entwickelt. Folgende Frameworks und Bibliotheken kommen zum Einsatz:

Jetpack Compose

Jetpack Compose wird als UI-Framework eingesetzt, da es vom Auftraggeber pep.digital vorgegeben wurde. Es verfolgt einen modernen deklarativen Ansatz, der die UI-Entwicklung vereinfacht, da die Benutzeroberfläche direkt in Kotlin beschrieben werden kann.

Glance API

Für die Entwicklung der Widgets kommt die Glance API zum Einsatz. Sie basiert auf Jetpack Compose und verfolgt ebenfalls einen deklarativen Ansatz, was die Entwicklung vereinfacht, da dieselben Konzepte wie im restlichen UI-Code verwendet werden können.

Retrofit 2

Die Netzwerkkommunikation mit dem Backend wird über Retrofit 2 erfolgen. Es ermöglicht, REST-Schnittstellen als Kotlin-Interfaces zu definieren, wodurch Netzwerkaufrufe kürzer und sicherer sind.

Backend

Das Backend wird mit Java und Spring Boot entwickelt und stellt die Schnittstellen für die Android-App bereit.

Spring Boot

Spring Boot wurde von der Hochschule als Backend-Framework vorgegeben und ermöglicht die Anbindung an die Datenbank, die Authentifizierung über Keycloak sowie die Bereitstellung der Schnittstellen für die Android-App.

Datenbanken

Die Datenhaltung erfolgt auf zwei Ebenen, um sowohl Offline-Nutzung als auch geräteübergreifenden Zugriff zu ermöglichen.

Room

Room dient als lokale Datenbank auf Basis von SQLite und ist die Single Source of Truth der Anwendung. Dies ermöglicht eine Offline-Nutzbarkeit. Sofern eine Internetverbindung vorhanden ist, läuft die Synchronisation mit dem Server nach dem Prinzip Local-First.

PostgreSQL

PostgreSQL wird serverseitig eingesetzt, um Nutzerdaten und Aktivitäten zentral zu speichern. Dadurch können die Daten geräteübergreifend abgerufen und langfristig gesichert werden.

Authentifizierung

Die Authentifizierung erfolgt zentral über Keycloak.

Keycloak

Für die Authentifizierung wird Keycloak eingesetzt. Es wurde von der Hochschule Esslingen vorgegeben und verwaltet die Benutzeridentitäten zentral und unabhängig von der Anwendungslogik.

Externe Schnittstellen

WeatherAPI.com

Über die WeatherAPI.com-Schnittstelle werden per HTTP-Aufruf die Wetterinformationen abgerufen. Die Kommunikation erfolgt über einen API-Key im Backend, welches wetterbasierte Push-Benachrichtigungen ermöglicht.

2.4 Verwendete Schnittstellentechnologie

Die Kommunikation zwischen Frontend, Backend und externen Diensten erfolgt über eine REST-basierte API. Die Daten werden im JSON-Format übertragen und über HTTP-Anfragen zwischen den Systemen ausgetauscht.

Für die Kommunikation zwischen Android-App und Backend wird im Frontend die Bibliothek Retrofit 2 verwendet. Das Spring-Boot-Backend stellt REST-Endpunkte bereit, über die Aktivitäten, Benutzerinformationen und Streak-Daten verarbeitet werden können.

Die Authentifizierung erfolgt über Keycloak mittels OAuth2/OIDC. Nach erfolgreichem Login erhält die mobile App ein Access-Token sowie ein langlebiges Refresh-Token. Während das Access-Token bei allen weiteren API-Anfragen im HTTP-Header zur Autorisierung mitgesendet wird, ermöglicht das Refresh-Token im Hintergrund eine automatische und nahtlose Erneuerung der Nutzersession, ohne dass eine erneute Passworteingabe erforderlich ist.

Zusätzlich kommuniziert das Backend per HTTPS mit WeatherAPI.com, um die Wetterdaten für die wetterbedingten Pushbenachrichtigungen zu erhalten.

Schnittstellenbeschreibung (API)

POST /api/users/login

- Anmeldung eines Nutzers
- Requested Body: { "email": "...", "password": "..." }
- Antwort: { "accessToken": "...", "userId": "UUID", "refreshToken": "..." }

POST /api/users/refresh

- Sitzungs Überprüfung
- Requested Body: { "refreshToken": "..." }
- Antwort: { "accessToken": "...", "userId": "UUID", "refreshToken": "..." }

POST /api/users/register

- Registrierung eines Nutzers
- Requested Body: { "name": "...", "vorname": "...",
"email": "...", "password": "..." }
- Antwort: { "userId": "..." }

POST /activities/upload

- Neue Aktivität speichern
- Requested Body: { "activityName": "Spazieren",


```
"activityDate": "2026-04-22" }
```

- Antwort: { "message": "Aktivität gespeichert" }

PUT /activities/id

- Bestehende Aktivität bearbeiten
- Requested Body: { "activityName": "Joggen" }
- Antwort: { "message": "Aktivität aktualisiert" }

GET /api/weather

- Wetterdaten abrufen
- Antwort: { "condition": "Sunny" }

2.5 Datenbank

2.5.1 Logisches Datenmodell

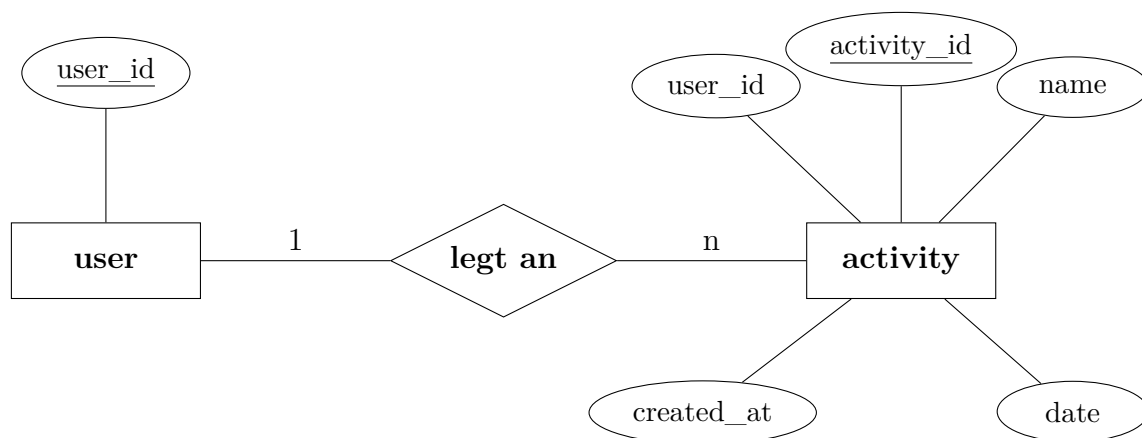
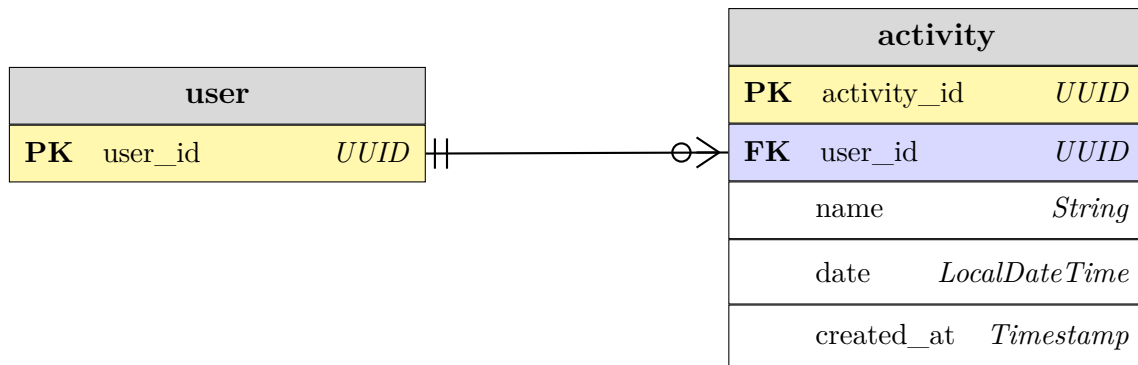


Abbildung 2.5: Konzeptionelles ER-Modell

2.5.2 Physisches Datenmodell



PK – Primary Key

FK – Foreign Key

Abbildung 2.6: Physisches Datenmodell

Anhang

Das folgende Glossar erläutert die in diesem Dokument verwendeten Fachbegriffe.

Begriff	Erklärung
MVP	Minimum Viable Product – minimaler funktionsfähiger Prototyp
IAM	Identity and Access Management – Verwaltung von Identitäten und Zugriffsrechten
OAuth2	Offenes Autorisierungsprotokoll für sicheren Zugriff auf Ressourcen
OIDC	OpenID Connect – Authentifizierungsschicht auf Basis von OAuth2
REST	Representational State Transfer – Architekturstil für Webservices
API	Application Programming Interface – Schnittstelle zur Kommunikation zwischen Systemen
Room	Googles offizielle Bibliothek für lokale Datenbanken auf Android, basierend auf SQLite
Glance	Jetpack-Bibliothek zur deklarativen Entwicklung von Android Home-Screen-Widgets

Quellen

1. Figma-Entwürfe, erstellt von Lionel Toussoumsnaba,(08.05.2026).. <https://www.figma.com/design/QJRfYFM1shPvySzpfwocuo/Activity-Tracker?node-id=0-1&t=z3Sr26iHMMWxgfBu-1>
2. WeatherAPI.com (08.05.2026).. <https://www.weatherapi.com>
3. Projekt-Repository auf GitHub (08.05.2026). <https://github.com/hse-classes/ss26-aktivitatstracker>